

PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERÚ

FACULTAD DE CIENCIAS E INGENIERÍA

INTELIGENCIA ARTIFICIAL (1INF24)

Examen 1 (Parte 1) – Semestre 2024-2

Indicaciones generales:

- Duración máxima: **60 minutos (1 hora)**.
 - Para responder las preguntas de esta parte **no está permitido ningún material**
 - **Solo se permite Editor de Texto** para responder las preguntas
 - Cualquier indicio de plagio resultará en la anulación de la prueba.
 - El archivo de respuestas nómbrelo **Examen1-Parte1_código.docx**, donde código es su código PUCP de 8 dígitos. Subirlo a Paideia antes de finalizar
 - Se tomará en cuenta en la calificación la precisión de la respuesta y el uso adecuado del lenguaje.
-

Pregunta 1 (2 puntos)

Compare BFS y DFS en términos de optimalidad, complejidad espacial y temporal. Discuta en qué escenarios uno es preferible sobre el otro.

 Respuesta – Pregunta 1

Criterio	BFS (Búsqueda en Amplitud)	DFS (Búsqueda en Profundidad)
Estrategia	Explora nivel por nivel desde la raíz (cola FIFO).	Explora un camino completo antes de retroceder (pila LIFO).
Optimalidad	✅ Óptima si todos los costos de acción son iguales (encuentra el camino más corto).	❌ No óptima , puede encontrar soluciones más largas antes que las más cortas.
Complejitud	✅ Completa si el factor de ramificación (b) es finito.	⚠️ Completa solo si el espacio de búsqueda es finito y no hay ciclos infinitos.
Complejidad temporal	$O(b^d)$ — crece exponencialmente con la profundidad de la solución.	$O(b^m)$ — puede ser grande si la profundidad máxima (m) $\gg d$.
Complejidad espacial	$O(b^d)$ — requiere mucha memoria (mantiene todos los nodos del nivel actual).	$O(b \cdot m)$ — bajo consumo de memoria , solo guarda el camino actual.

Tipo de Cola (FIFO).
estructura usada

Pila (LIFO).

Cuándo usar cada uno

- **Usar BFS cuando:**
 - Se necesita una **solución óptima o mínima en pasos**.
 - El espacio de búsqueda es **pequeño o moderado** (porque su costo de memoria crece rápido).
 - Ejemplo: búsqueda de rutas más cortas o juegos con pocos movimientos posibles.
- **Usar DFS cuando:**
 - El espacio de búsqueda es **muy grande** o la **memoria es limitada**.
 - No se requiere la solución óptima, solo **una solución cualquiera rápidamente**.
 - Ejemplo: exploración de laberintos grandes o validación de existencia de soluciones.

En resumen:

- **BFS** = óptima y completa, pero costosa en memoria.
- **DFS** = eficiente en memoria, pero no garantiza la mejor solución.
👉 Por tanto, **BFS se prefiere para soluciones óptimas en espacios pequeños**, y **DFS para exploraciones profundas o problemas de existencia**.

Pregunta 2 (2 puntos)

Explique por qué una heurística inadmisibile no garantiza optimalidad en el algoritmo A*

Qué significa que una heurística sea inadmisibile

En A*, la heurística $h(n)$ intenta **adivinar cuánto falta** para llegar al objetivo desde un punto n .

- Si $h(n)$ no se pasa del costo real, se dice que es **admisible** → es “optimista”.
- Si $h(n)$ se pasa (sobreestima), se dice que es **inadmisible** → es “demasiado confiada o exagerada”.

⚙️ Cómo trabaja A*

A* decide qué camino explorar según esta fórmula:

$$f(n)=g(n)+h(n) \quad f(n) = g(n) + h(n) \quad f(n)=g(n)+h(n)$$

- $g(n)$ = lo que ya me costó llegar hasta aquí.
- $h(n)$ = lo que creo que me falta.

A* siempre expande el nodo con **menor $f(n)$** (el camino que parece más barato).

⚠️ Qué pasa si la heurística es inadmisibile

Si $h(n)$ sobreestima (es decir, dice que falta más de lo que realmente cuesta), entonces A* puede pensar que **un camino bueno parece caro** y lo **descarta demasiado pronto**.

Por eso, podría terminar eligiendo **otro camino más largo**, pero que *parece* más barato por error.

💡 Ejemplo sencillo

Supón que vas de **A → B → C (meta)** y A* calcula:

Camino	Costo real total	$h(n)$ (estimado)	$f(n) = g+h$
A → B → C	10	12 (sobreestima)	15
A → D → C	13	1	14

Aunque el **camino A–B–C es mejor (10)**, A* cree que costará **15**, y elige **A–D–C (14)**. Así **pierde la solución óptima**.

✅ En resumen

- Una **heurística inadmisble sobreestima** el costo que falta.
- Esto hace que A^* **piense erróneamente** que el camino correcto es más caro de lo que es.
- Por eso, A^* puede **tomar una ruta subóptima** y **ya no garantiza la mejor solución posible**.

Pregunta 3 (2 puntos)

Explique brevemente en qué consiste el método del problema relajado para generar heurísticas y por qué las heurísticas que se generan con este método son admisibles en el problema original

Método del problema relajado:

Consiste en **simplificar o eliminar una o más restricciones del problema original**, de modo que el nuevo problema (relajado) sea **más fácil de resolver**.

Luego, el **costo de la solución óptima del problema relajado** se usa como **heurística $h(n)$** para el problema original.

Ejemplo:

En el **8-puzzle**, si relajamos la restricción de mover solo fichas adyacentes, podemos estimar el costo contando simplemente cuántas fichas están fuera de lugar.

Por qué las heurísticas generadas así son admisibles

Porque al quitar restricciones, el problema relajado **nunca puede costar más que el real** —es decir, su solución siempre será **igual o más barata** que la del problema original:

$$h(n) \leq h^*(n) \leq h(n) \leq h^*(n)$$

Por eso, la heurística derivada del problema relajado **nunca sobreestima** el costo verdadero y, por tanto, es **admisble**.

En resumen:

- El problema relajado **simplifica reglas** para estimar más fácilmente.
- Su solución óptima da una **cota inferior** del costo real.
- Las heurísticas obtenidas **siempre son admisibles** en el problema original.

Pregunta 4 (2 puntos)

Especifique una función de enfriamiento que haga que Simulated Annealing tenga el mismo comportamiento que Hill Climbing. Discuta también cómo ajustar la función de enfriamiento de Simulated Annealing para controlar el balance exploración/explotación, y cómo esto afecta la rapidez de convergencia y optimalidad de las soluciones.

Primera parte:

¿Qué función de enfriamiento haría que SA se comporte igual que Hill Climbing?

👉 En *Simulated Annealing*, la **temperatura (T)** controla la probabilidad de aceptar soluciones peores.

Esa probabilidad es:

$$P = e^{-\Delta E / T} = e^{\{-\Delta E / T\}} P = e^{-\Delta E / T}$$

donde ΔE es el aumento del costo (empeoramiento).

- Si **T es muy alta**, SA **acepta soluciones peores con frecuencia** → exploración.
- Si **T es muy baja**, SA **rechaza casi todas las soluciones peores** → se comporta como Hill Climbing.

Por tanto:

Si la temperatura $T = 0$ (o decrece inmediatamente a 0 desde el inicio), SA se comporta exactamente como Hill Climbing,
porque nunca acepta un movimiento peor.

💡 Ejemplo:

Función de enfriamiento:

$T(k) = 0$ o $T(k) = e^{-k}$ (cae bruscamente al inicio) $T(k) = 0 \quad \text{o} \quad T(k) = e^{-k}$ (cae bruscamente al inicio)

produce el mismo comportamiento que Hill Climbing: **solo acepta mejoras**.

⚙️ Segunda parte: Cómo ajustar la función de enfriamiento para controlar el balance exploración/explotación

El **balance** entre explorar (probar nuevas zonas) y explotar (refinar la mejor zona) depende de cómo disminuye T:

Tipo de enfriamiento	Efecto	Comportamiento
----------------------	--------	----------------

Rápido ($T \downarrow$ muy rápido) Menos exploración, converge más rápido pero puede quedar en óptimos locales. ♦ Más parecido a Hill Climbing.

Lento gradual ($T \downarrow$) Mayor exploración al inicio, evita óptimos locales, converge más lento. ♦ Mayor probabilidad de alcanzar el óptimo global.

💡 Ejemplos de funciones de enfriamiento:

- **Lineal:** $T(k) = T_0 - \alpha k$
- **Exponencial:** $T(k) = T_0 \cdot e^{-\alpha k}$
- **Logarítmica (muy lenta):** $T(k) = \frac{T_0}{\log(k+1)}$

⚖️ Tercera parte: Efecto sobre rapidez y optimalidad

Enfriamiento	Exploración	Rapidez de convergencia	Optimalidad esperada
Muy rápido	Baja	Alta (rápida convergencia)	Baja (posible estancamiento local)
Moderado	Equilibrada	Media	Buena
Muy lento	Alta	Lenta (más iteraciones)	Alta (mayor probabilidad de óptimo global)

✅ Resumen final

- **SA = Hill Climbing** si $T = 0$ o **enfria instantáneamente** → no acepta soluciones peores.
- Para **controlar exploración/explotación**, ajusta la **tasa de enfriamiento (α)**:
 - **Alta α** → **rápida convergencia, riesgo de óptimos locales.**
 - **Baja α** → **búsqueda más amplia, mayor chance de óptimo global.**
- **Compromiso:** un enfriamiento **lento pero no infinito** logra buen equilibrio entre **rapidez** y **calidad** de solución.

Pregunta 5 (2 puntos)

En los algoritmos mostrados abajo, indique de forma precisa en qué línea(s) se intercambia información social entre partículas en PSO y entre fuentes de alimento en ABC. De una breve explicación en cada caso.

PSO

```
1:  $S \leftarrow \text{SwarmInitialization}()$ 
2: while not stop condition do
3:   for each particle  $x_i$  of the swarm  $S$  do
4:     evaluate( $x_i$ )
5:     if  $\text{fitness}(x_i)$  is better than  $\text{fitness}(pBest_i)$  then
6:        $pBest_i \leftarrow x_i$ 
7:     end if
8:     if  $\text{fitness}(pBest_i)$  is better than  $\text{fitness}(g)$  then
9:        $g \leftarrow pBest_i$ 
10:    end if
11:  end for
12:  for each particle  $x_i$  of the swarm  $S$  do
13:     $v_i^{k+1} = \omega \cdot v_i^k + \varphi_1 \cdot rnd_1 \cdot (pBest_i - x_i^k) + \varphi_2 \cdot rnd_2 \cdot (g - x_i^k)$ 
14:     $x_i^{k+1} = x_i^k + v_i^{k+1}$ 
15:  end for
16: end while
17: Output: best solution found
```

PSO:

Primero recordemos cómo funciona PSO

Cada partícula tiene:

- $x_i \rightarrow$ su posición actual (solución actual).
- $pBest_i \rightarrow$ su mejor posición personal (memoria individual).
- $gBest \rightarrow$ la mejor posición global encontrada por todo el enjambre (información social).

Pseudocódigo y líneas clave

Del algoritmo que muestras:

```

1: S ← SwarmInitialization()
...
5-6: if fitness(xi) is better than fitness(pBesti) then
    pBesti ← xi
...
7-8: if fitness(pBesti) is better than fitness(g) then
    g ← pBesti
...
13: vi{k+1} = ω · vik + φ1 · rnd1 · (pBesti - xik) + φ2 · rnd2 · (g - xik)
14: xi{k+1} = xik + vi{k+1}

```

Línea(s) donde se intercambia información social

Línea 8:

`if fitness(pBesti) is better than fitness(g) then g ← pBesti`

Aquí cada partícula comparte su mejor posición (**pBest_i**) con el enjambre, y el enjambre actualiza la **mejor posición global g**.

👉 Este es el **intercambio social de información**: las partículas “aprenden” unas de otras a través del valor global **g**.

Línea 13:

`vi{k+1} = ωvik + φ1rnd1(pBesti - xik) + φ2rnd2(g - xik)`

En esta ecuación, cada partícula **usa la información global g** (social) y su propia mejor posición (cognitiva) para actualizar su velocidad.

👉 Aquí se **aplica la influencia social** en el movimiento.

Explicación breve

- En **línea 8**, ocurre la **difusión de la mejor solución global (g)** a todas las partículas → “*aprendizaje social*”.
- En **línea 13**, esa información social se **utiliza** para ajustar la dirección y velocidad de cada partícula → “*seguimiento colectivo hacia el mejor individuo*”.

✓ Resumen PSO

Tipo de información	Línea	Qué sucede
Social (gBest)	8	Se actualiza el mejor global del enjambre.
Aplicación social	13	Cada partícula ajusta su velocidad usando gBest (aprendizaje social).

ABC

```
(1) Generate the initial population  $x_i$  ( $i=1,2,\dots,SN$ )
(2) Evaluate the fitness( $fit(x_i)$ ) of the population
(3)  $g = 0$ 
(4) Repeat
(5)   For each employed bee{
        Produce new solutions  $v_{i,g}$  in the neighbourhood of  $x_i$ 
        Calculate its fitness value  $fit(v_{i,g})$ 
        Apply the selection process between  $x_i$  and  $v_{i,g}$  }
(6)   For each solution ( $x_i$ ){ Calculate the probability values  $p_i = \frac{fit_i}{\sum_{n=1}^{SN} fit_n}$  }
(7)   For each onlooker bee{
        Select a solution  $x_i$  depending on  $p_i$ 
        Produce new solutions  $v_{i,g}$  from the solutions  $x_i$ 
        Calculate its fitness value  $fit(v_{i,g})$ 
        Apply the selection process }
(8)   If there is an abandoned solution  $x_i$  then
        replace it with a new randomly produced solution  $x_i$  by the scout bee
        Memorize the best solution achieved so far
(9)    $g = g + 1$ 
      Until  $g = \text{max\_iter}$ 
```

ABC:

🧠 Recordemos cómo funciona ABC

En el algoritmo ABC hay tres tipos de abejas:

- **Empleadas (employed bees):** exploran alrededor de sus fuentes actuales.
- **Observadoras (onlooker bees):** eligen fuentes basadas en su calidad (fitness).

- **Exploradoras (scouts):** reemplazan fuentes estancadas con nuevas aleatorias.

El **intercambio social** ocurre cuando las abejas **comparten la información de la calidad de las fuentes de alimento** para guiar la búsqueda colectiva.

⚙️ Pseudocódigo (fragmentos relevantes)

(5) For each employed bee

 Produce new solutions ...

 Calculate fitness ...

 Apply the selection process ...

(6) For each solution (x_i)

 Calculate probability values $p_i = \text{fit}_i / \sum \text{fit}$

(7) For each onlooker bee

 Select a solution x_i depending on p_i

 Produce new solutions ...

 Calculate fitness ...

 Apply the selection process

(8) If there is an abandoned solution ...

📌 Líneas donde ocurre el intercambio social

➡️ Línea (6):

Calculate probability values $p_i = \text{fit}_i / \sum \text{fit}$

Aquí se **difunde la información social**: cada fuente de alimento (solución) comunica su calidad (**fitness**) al resto de la colonia mediante su **probabilidad p_i** .

👉 Es el **mecanismo de comunicación indirecta** (“**danza de las abejas**”), donde se informa qué fuentes son mejores.

➡️ Línea (7):

Each onlooker bee selects a solution x_i depending on p_i

Aquí se **usa la información social**: las **abejas observadoras** eligen las fuentes con mayor probabilidad (mejor calidad) y exploran sus alrededores.

👉 Es el **aprendizaje social** propiamente dicho: las abejas deciden basadas en lo que otras encontraron.

Explicación breve

Fase	Línea	Qué sucede	Tipo de información
Cálculo de probabilidades (danza)	(6)	Se comparte la calidad de las fuentes con toda la colonia.	Difusión social (comunicación indirecta).
Selección por observadoras	(7)	Las abejas eligen fuentes de alimento en base a la información compartida (p_i).	Intercambio y uso de información social.

Resumen ABC

- **Línea 6:** ocurre la **comunicación social** (las abejas publican su fitness).
- **Línea 7:** ocurre el **uso de la información social** (las observadoras seleccionan fuentes basadas en esa información).

En palabras simples:

En PSO las partículas comparten directamente su mejor posición ($gBest$), mientras que en ABC las abejas **comunican de forma indirecta** la calidad de sus fuentes (vía probabilidades p_i), y otras **deciden en base a esa información**.